

CPSC 304 Project Cover Page

Milestone #: 2

Date: February 24, 2026

Group Number: 107

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Justin Hoang	94126356	t3v8j	Just.hoang@outlook.com
Hao Hu	12260022	t9s7z	helenhaohu@gmail.com
Angel Shen	98425549	i0n5o	an.shen222@gmail.com

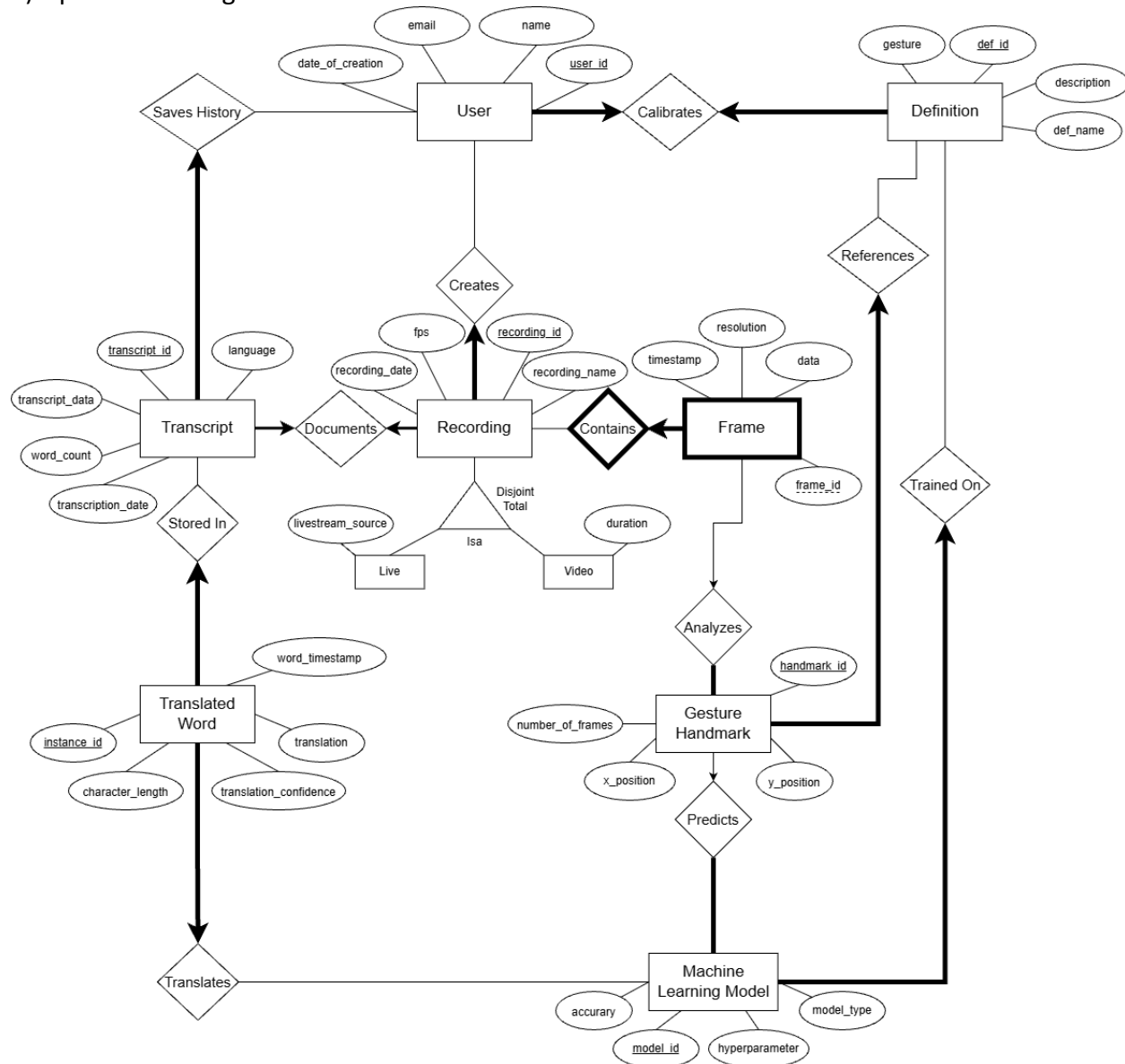
By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

2) Our project combines computer vision and image capture to translate sign-language into words. We want to be able to take in either a video or live footage and translate the alphabet from American sign language to English text using machine learning and databases.

3) We'll be using Python for the backend and machine learning, React for the frontend, and Oracle for the database.

4) Updated ER Diagram:



University of British Columbia, Vancouver

Department of Computer Science

5) Schema:

PK: Primary Key, CK: Candidate Key, FK: Foreign Key

Calibrated_User (user_id: integer, def_id: integer, date_of_creation: date, email: varchar, name: varchar)

- PK: user_id
- CK:
 - o email
 - o def_id
- FK: def_id from Calibrated_Definition(def_id)
- UNIQUE email
 - o Each user must have a different email address
- NOT NULL email, date_of_creation
 - o A user cannot exist without an email
 - o Every user must have an account creation date, as it comes with creation

Calibrated_Definition(def_id: integer, user_id: integer, gesture: varchar, def_name: varchar, description: varchar)

- PK: def_id
- CK:
 - o def_name, user_id
 - o gesture, user_id
- FK: user_id from Calibrated_User(user_id)
- NOT NULL gesture, def_name
 - o A definition must have a gesture and a def_name, as that is what is used for mapping and identification

Created_Documented_Recording(recording_id: integer, transcript_id: integer, user_id: integer, recording_name: varchar, fps: integer, recording_date: date)

- PK: recording_id
- CK:
 - o recording_name
 - o transcript_id
- FK: transcript_id from Documented_Saved_Transcript(transcript_id), user_id from Calibrated_User(user_id)
- UNIQUE (recording_name, user_id)
 - o Each user can only create unique recording names
- NOT NULL: fps, recording_date
 - o Every recording must have a frame rate and a creation date

Contained_Analyzed_Frame(recording_id: integer, frame_id: integer, timestamp: timestamp, resolution: varchar, data: varchar)

- PK: recording_id, frame_id

- CK: recording_id, timestamp
- FK: recording_id from Created_Documented_Recording(recording_id)
- NOT NULL: timestamp, resolution, data
 - o A frame must occur at a particular time, must have a resolution, and must contain frame data
- frame_id is a partial key
- On deletion of a Recording, all owned Frames will also be deleted

Live(recording_id: integer, livestream_source: varchar)

- PK: recording_id
- FK: recording_id from Created_Documented_Recording(recording_id)
- NOT NULL: livestream_source
 - o There must be a source if the recording is a live recording

Video(recording_id: integer, duration: integer)

- PK: recording_id
- FK: recording_id from Created_Documented_Recording(recording_id)
- NOT NULL: duration
 - o A stored video must have a duration

Documented_Saved_Transcript(transcript_id: integer, recording_id: integer, user_id: integer, transcript_data: varchar, language: varchar, word_count: integer, transcription_date: date)

- PK: transcript_id
- CK: recording_id
- FK: recording_id from Created_Documented_Recording(recording_id), user_id from Calibrated_User(user_id)
- NOT NULL: transcript_data, transcription_date, language
 - o A transcript must contain data (the transcript text), each transcript comes with a date of creation, and each transcript must have its own language

Stored_Translated_Word(model_id: integer, transcript_id: integer, instance_id: integer, handmark_id: integer, character_length: integer, translation_confidence: integer, translation: varchar, word_timestamp: timestamp)

- PK: instance_id
- FK: transcript_id from Documented_Saved_Transcript(transcript_id), handmark_id from Predicted_Gesture_Handmark(handmark_id), model_id from Trained_Machine_Learning_Model(model_id)
- NOT NULL translation
 - o A stored translated word must contain actual translated text, otherwise there is no point in storing it

Predicted_Gesture_Handmark(handmark_id: integer, def_id: integer, model_id: integer, number_of_frames: integer, x_position: varchar, y_position: varchar)

- PK: handmark_id
- CK: number_of_frames, x_position, y_position
- FK: model_id from Trained_Machine_Learning_Model(model_id)
- UNIQUE x_position, y_position
 - o Unique x_position and y_position is how gestures will be identified using the machine model
- NOT NULL number_of_frames, x_position, y_position
 - o Similarly, a gesture must span some number of frames and have coordinates, otherwise it is meaningless

Trained_Machine_Learning_Model(model_id: integer, handmark_id: integer, accuracy: integer, hyperparameter: integer, model_type: varchar)

- PK: model_id
- CK: model_type, hyperparameter
- FK: handmark_id from Predicted_Gesture_Handmark(handmark_id)
- NOT NULL model_type, hyperparameter
 - o Model must have a type, and must store configuration to be used

6) FD: Functional Dependencies

Calibrated_User (user_id: integer, def_id: integer, date_of_creation: date, email: varchar, name: varchar)

- FD:
 - user_id -> def_id, date_of_creation, email, name
 - email -> user_id, def_id, date_of_creation, name
 - def_id -> user_id, date_of_creation, name, email

Calibrated_Definition(def_id: integer, user_id: integer, gesture: varchar, def_name: varchar, description: varchar)

- FD:
 - def_id -> user_id, gesture, def_name, description
 - def_name, user_id -> def_id, gesture, description
 - gesture, user_id -> def_name, def_id, description

Created_Documented_Recording(recording_id: integer, transcript_id: integer, user_id: integer, recording_name: varchar, fps: integer, recording_date: date)

- FD:
 - recording_id -> transcript_id, user_id, recording_name, fps, recording_date
 - recording_name -> recording_id, transcript_id, user_id, fps, recording_date
 - transcript_id -> recording_id, user_id, recording_name, fps, recording_date

Contained_Analyzed_Frame(recording_id: integer, frame_id: integer, timestamp: timestamp, resolution: varchar, data: varchar)

- FD:
 - recording_id, frame_id -> timestamp, resolution, data
 - recording_id, timestamp -> frame_id, resolution, data
 - data -> resolution

Live(recording_id: integer, livestream_source: varchar)

- FD:
 - recording_id -> livestream_source

Video(recording_id: integer, duration: integer)

- FD:
 - recording_id -> duration

Documented_Saved_Transcript(transcript_id: integer, recording_id: integer, user_id: integer, transcript_data: varchar, language: varchar, word_count: integer, transcription_date: date)

- FD:
 - transcript_id -> recording_id, user_id, transcript_data, word_count, transcription_date, language

- recording_id -> transcript_id, user_id, transcript_data, word_count, transcription_date, language
- transcript_data -> word_count, language

Stored_Translated_Word(model_id: integer, transcript_id: integer, instance_id: integer, handmark_id: integer, character_length: integer, translation_confidence: number, translation: varchar, word_timestamp: timestamp)

- FD:
 - instance_id -> transcript_id, handmark_id, character_length, translation_confidence, translation, model_id
 - translation -> character_length
 - model_id, handmark_id, translation -> translation_confidence
 - handmark_id -> word_timestamp

Predicted_Gesture_Handmark(handmark_id: integer, def_id: integer, model_id: integer, number_of_frames: integer, x_position: varchar, y_position: varchar)

- FD:
 - handmark_id -> def_id, model_id, number_of_frames, x_position, y_position
 - number_of_frames, x_position, y_position -> handmark_id, def_id, model_id
 - def_id -> x_position, y_position, number_of_frames

Trained_Machine_Learning_Model(model_id: integer, handmark_id: integer, accuracy: integer, hyperparameter: integer, model_type: varchar)

- FD:
 - model_id -> handmark_id, accuracy, hyperparameter, model_type
 - hyperparameter, model_type -> model_id, handmark_id, accuracy

7) Normalizing Tables

Normally, we would list the primary and foreign keys, but since in this process we will be creating many new tables, we decided to use underlining to show a primary key and bolding to show a foreign key.

Calibrated_User (user_id: integer, def_id: integer, date_of_creation: date, email: varchar, name: varchar)

- FD:
 - o user_id -> def_id, date_of_creation, email, name
 - o email -> user_id, def_id, date_of_creation, name
 - o def_id -> user_id, date_of_creation, name, email
- Normalization:
 - o All FD's have primary/candidate keys on the LHS, so the table is in BCNF and 3NF
- Keys:
 - o PK: user_id
 - o CK:
 - email
 - def_id
 - o FK: def_id from Calibrated_Definition(def_id)

Calibrated_Definition(def_id: integer, user_id: integer, gesture: varchar, def_name: varchar, description: varchar)

- FD:
 - o def_id -> user_id, gesture, def_name, description
 - o def_name, user_id -> def_id, gesture, description
 - o gesture, user_id -> def_name, def_id, description
- Normalization:
 - o All FD's have primary/candidate keys on the LHS, so the table is in BCNF and 3NF
- Keys:
 - o PK: def_id
 - o CK:
 - def_name, user_id
 - gesture, user_id
 - o FK: user_id from Calibrated_User(user_id)

Created_Documented_Recording(recording_id: integer, transcript_id: integer, user_id: integer, recording_name: varchar, fps: integer, recording_date: date)

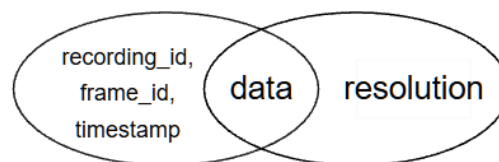
- FD:
 - o recording_id -> transcript_id, user_id, recording_name, fps, recording_date
 - o recording_name -> recording_id, transcript_id, user_id, fps, recording_date
 - o transcript_id -> recording_id, user_id, recording_name, fps, recording_date
- Normalization:

- All FD's have primary/candidate keys on the LHS, so the table is in BCNF and 3NF
- Keys:
 - PK: recording_id
 - CK:
 - recording_name
 - transcript_id
 - FK: transcript_id from Documented_Saved_Transcript(transcript_id), user_id from Calibrated_User(user_id)

Contained_Analyzed_Frame(recording_id: integer, frame_id: integer, timestamp: timestamp, resolution: varchar, data: varchar)

- FD:
 - recording_id, frame_id -> timestamp, resolution, data
 - recording_id, timestamp -> frame_id, resolution, data
 - data -> resolution
- Normalization into BCNF
 - Closures:
 - {recording_id, frame_id}⁺ = {recording_id, frame_id, timestamp, resolution, data}
 - {recording_id, timestamp}⁺ = {recording_id, timestamp, frame_id, resolution, data}
 - {data}⁺ = {data, resolution}
 - Decompose:
 - Since recording_id, frame_id and recording_id, timestamp are both keys, decompose on data -> resolution

Decompose on data -> resolution



- - `Contained_Analyzed_Frame1(data, recording_id, frame_id, timestamp)`,
`Contained_Analyzed_Frame2(data, resolution)`
- Keys:
 - `Contained_Analyzed_Frame1(data, recording_id, frame_id, timestamp)`
 - PK: recording_id, frame_id
 - CK: recording_id, timestamp
 - FK:
 - recording_id from `Created_Documented_Recording(recording_id)`
 - data from `Contained_Analyzed_Frame2(data, resolution)`
 - `Contained_Analyzed_Frame2(data, resolution)`

- PK: data

Live(recording_id: integer, livestream_source: varchar)

- FD:
 - recording_id -> livestream_source
- Normalization
 - All FD's have primary/candidate keys on the LHS, so the table is in BCNF and 3NF
- Keys:
 - PK: recording_id
 - FK: recording_id from Created_Documented_Recording(recording_id)

Video(recording_id: integer, duration: integer)

- FD:
 - recording_id -> duration
- Normalization
 - All FD's have primary or candidate keys on the LHS, so the table is in BCNF
- Keys:
 - PK: recording_id
 - FK: recording_id from Created_Documented_Recording(recording_id)

Documented_Saved_Transcript(transcript_id: integer, recording_id: integer, user_id: integer, transcript_data: varchar, language: varchar, word_count: integer, transcription_date: date)

- FD:
 - transcript_id -> recording_id, user_id, transcript_data, word_count, transcription_date, language
 - recording_id -> transcript_id, user_id, transcript_data, word_count, transcription_date, language
 - transcript_data -> word_count, language
- Normalization
 - Closures:
 - {transcript_id}⁺ = {recording_id, user_id, transcript_data, word_count, transcription_date, language}
 - {recording_id}⁺ = {transcript_id, user_id, transcript_data, word_count, transcription_date, language}
 - {transcript_data}⁺ = {transcript_data, word_count, language}
 - {user_id}⁺ = {user_id}
 - {language}⁺ = {language}
 - {word_count}⁺ = {word_count}
 - {transcription_date}⁺ = {transcription_date}
 - Minimal key is transcript_id and recording_id
 - Other than the first two FDs, none of the other FDs have a RHS that is part of a minimal key or a LHS that is a superkey
 - Split RHS:

- $\text{transcript_id} \rightarrow \text{recording_id}$
- $\text{transcript_id} \rightarrow \text{user_id}$
- $\text{transcript_id} \rightarrow \text{transcript_data}$
- $\text{transcript_id} \rightarrow \text{word_count}$
- $\text{transcript_id} \rightarrow \text{transcription_date}$
- $\text{transcript_id} \rightarrow \text{language}$
- $\text{recording_id} \rightarrow \text{transcript_id}$
- $\text{recording_id} \rightarrow \text{user_id}$
- $\text{recording_id} \rightarrow \text{transcript_data}$
- $\text{recording_id} \rightarrow \text{word_count}$
- $\text{recording_id} \rightarrow \text{transcription_date}$
- $\text{recording_id} \rightarrow \text{language}$
- $\text{transcript_data} \rightarrow \text{word_count}$
- $\text{transcript_data} \rightarrow \text{language}$
- This is now a minimal cover because the LHS is minimized, and none of the FD's are redundant.
- Create a relation for every FD in the minimal cover:
 - $\text{Documented_Saved_Transcript_1}(\underline{\text{transcript_id}}$: integer, recording_id : integer)
 - $\text{Documented_Saved_Transcript_2}(\underline{\text{transcript_id}}$: integer, user_id : integer)
 - $\text{Documented_Saved_Transcript_3}(\underline{\text{transcript_id}}$: integer, transcript_data : varchar)
 - $\text{Documented_Saved_Transcript_4}(\underline{\text{transcript_id}}$: integer, word_count : integer)
 - $\text{Documented_Saved_Transcript_5}(\underline{\text{transcript_id}}$: integer, $\text{transcription_date}$: date)
 - $\text{Documented_Saved_Transcript_6}(\underline{\text{transcript_id}}$: integer, language : varchar)
 - $\text{Documented_Saved_Transcript_7}(\underline{\text{recording_id}}$: integer, $\underline{\text{transcript_id}}$: integer)
 - $\text{Documented_Saved_Transcript_8}(\underline{\text{recording_id}}$: integer, user_id : integer)
 - $\text{Documented_Saved_Transcript_9}(\underline{\text{recording_id}}$: integer, transcript_data : varchar)
 - $\text{Documented_Saved_Transcript_10}(\underline{\text{recording_id}}$: integer, word_count : integer)
 - $\text{Documented_Saved_Transcript_11}(\underline{\text{recording_id}}$: integer, $\text{transcription_date}$: date)
 - $\text{Documented_Saved_Transcript_12}(\underline{\text{recording_id}}$: integer, language : varchar)
 - $\text{Documented_Saved_Transcript_13}(\underline{\text{transcript_data}}$: varchar, word_count : integer)
 - $\text{Documented_Saved_Transcript_14}(\underline{\text{transcript_data}}$: varchar, language : varchar)

Stored_Translated_Word(model_id: integer, transcript_id: integer, instance_id: integer, handmark_id: integer, character_length: integer, translation_confidence: number, translation: varchar, word_timestamp: timestamp)

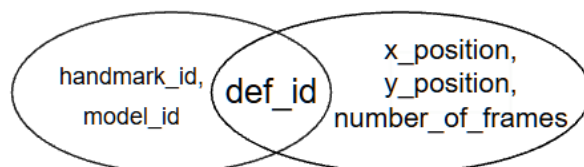
- FD:
 - o instance_id -> transcript_id, handmark_id, character_length, translation_confidence, translation, model_id
 - o translation -> character_length
 - o model_id, handmark_id, translation -> translation_confidence
 - o handmark_id -> word_timestamp
- Normalization into 3NF
 - o {instance_id}⁺ = {transcript_id, handmark_id, character_length, translation_confidence, translation, model_id}
 - o {translation}⁺ = {character_length}
 - o {model_id, handmark_id, translation}⁺ = {translation_confidence}
 - o {handmark_id}⁺ = {word_timestamp}
 - o Minimal key is instance_id
 - o Checking if it is already in 3NF:
 - Except for the first FD, none of the FDs have a RHS that is part of a minimal key
 - o Converting to 3NF using synthesis:
 - Simplify RHS:
 - instance_id -> transcript_id
 - instance_id -> handmark_id
 - instance_id -> character_length
 - instance_id -> translation_confidence
 - instance_id -> translation
 - instance_id -> model_id
 - translation -> character_length
 - model_id, handmark_id, translation -> translation_confidence
 - handmark_id -> word_timestamp
 - This is now a minimal cover because the LHS is minimized, and none of the FD's are redundant. We can now create our new relations
 - Translated_Word_1 (instance_id: integer, **transcript_id**: integer)
 - Translated_Word_2 (**instance_id**: integer, **handmark_id**: integer)
 - Translated_Word_3 (**instance_id**: integer, character_length: integer)
 - Translated_Word_4 (**instance_id**: integer, translation_confidence: integer)
 - Translated_Word_5 (**instance_id**: integer, **translation**: varchar)
 - Translated_Word_6 (**instance_id**: integer, **model_id**: integer)
 - Translated_Word_7 (translation: varchar, character_length: integer)

- Translated_Word_8 (model_id: integer, handmark_id: integer, translation: varchar, translation_confidence: integer)
- Translated_Word_9 (handmark_id: integer, word_timestamp: timestamp)
- These contain a key, so no new relation is needed

Predicted_Gesture_Handmark(handmark_id: integer, def_id: integer, model_id: integer, number_of_frames: integer, x_position: varchar, y_position: varchar)

- FD:
 - $\text{handmark_id} \rightarrow \text{def_id}, \text{model_id}, \text{number_of_frames}, \text{x_position}, \text{y_position}$
 - $\text{number_of_frames}, \text{x_position}, \text{y_position} \rightarrow \text{handmark_id}, \text{def_id}, \text{model_id}$
 - $\text{def_id} \rightarrow \text{x_position}, \text{y_position}, \text{number_of_frames}$
- Normalization into BCNF
 - Closures:
 - $\{\text{handmark_id}\}^+ = \{\text{handmark_id}, \text{def_id}, \text{model_id}, \text{number_of_frames}, \text{x_position}, \text{y_position}\}$
 - $\{\text{number_of_frames}, \text{x_position}, \text{y_position}\}^+ = \{\text{number_of_frames}, \text{x_position}, \text{y_position}, \text{handmark_id}, \text{def_id}, \text{model_id}\}$
 - $\{\text{def_id}\}^+ = \{\text{def_id}, \text{x_position}, \text{y_position}, \text{number_of_frames}\}$
 - Decompose:
 - Since handmark_id is a primary key and number_of_frames, x_position, y_position makes up a candidate key, we only need to decompose $\text{def_id} \rightarrow \text{x_position}, \text{y_position}, \text{number_of_frames}$

Decompose on
 $\text{def_id} \rightarrow \text{x_position}, \text{y_position}, \text{number_of_frames}$



-
- Predicted_Gesture_Handmark1(handmark_id: integer, def_id: integer, model_id: integer)
- Predicted_Gesture_Handmark2(def_id: integer, number_of_frames: integer, x_position: varchar, y_position: varchar)
- Keys:
 - Predicted_Gesture_Handmark1(handmark_id: integer, def_id: integer, model_id: integer)
 - PK: handmark_id
 - FK:
 - model_id from Trained_Machine_Learning_Model(model_id)

- def_id from Predicted_Gesture_Handmark2(def_id: integer, number_of_frames: integer, x_position: varchar, y_position: varchar)
- Predicted_Gesture_Handmark2(def_id: integer, number_of_frames: integer, x_position: varchar, y_position: varchar)
 - PK: def_id

Trained_Machine_Learning_Model(model_id: integer, handmark_id: integer, accuracy: integer, hyperparameter: integer, model_type: varchar)

- FD:
 - Model_id -> handmark_id, accuracy, hyperparameter, model_type
 - hyperparameter, model_type -> model_id, handmark_id, accuracy
- Normalization
 - All FD's have primary/candidate keys on the LHS, so the table is in BCNF and 3NF
- Keys:
 - PK: model_id
 - CK: model_type, hyperparameter
 - FK: handmark_id from Predicted_Gesture_Handmark(handmark_id)

8) SQL DDL

```
CREATE TABLE Calibrated_User(  
    user_id INT PRIMARY KEY,  
    def_id INT,  
    date_of_creation DATE NOT NULL,  
    email VARCHAR[50] NOT NULL,  
    name VARCHAR[50],  
    UNIQUE(email),  
    FOREIGN KEY(def_id) REFERENCES Calibrated_Definition(def_id))
```

Deleting a definition does not logically require deleting the user. Therefore, ON DELETE does not need to cascade.

```
CREATE TABLE Calibrated_Definition(  
    def_id INT PRIMARY KEY,  
    user_id INT,  
    gesture VARCHAR[255] NOT NULL,  
    def_name VARCHAR[50] NOT NULL,  
    description VARCHAR[255]  
    FOREIGN KEY (user_id) REFERENCES Calibrated_User (user_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the user_id foreign key because a definition cannot exist without the user who created it.

```
CREATE TABLE Created_Documented_Recording(  
    recording_id INT PRIMARY KEY,  
    transcript_id INT,  
    user_id INT,  
    fps INT NOT NULL,  
    recording_date DATE NOT NULL,  
    recording_name VARCHAR[50] NOT NULL,  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id),  
    FOREIGN KEY (user_id) REFERENCES Calibrated_User (user_id)  
    ON DELETE CASCADE,  
    UNIQUE(recording_name))
```

We used ON DELETE CASCADE for the user_id foreign key because a Recording cannot exist without the user who created it.

```
CREATE TABLE Contained_Analyzed_Frame1(  
    frame_id INT PRIMARY KEY,  
    recording_id INT,  
    user_id INT,  
    frame_number INT NOT NULL,  
    frame_data VARCHAR[255] NOT NULL,  
    FOREIGN KEY (recording_id) REFERENCES Created_Documented_Recording(recording_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (user_id) REFERENCES Calibrated_User (user_id)  
    ON DELETE CASCADE,  
    UNIQUE(frame_id, frame_number))
```

```
data VARCHAR[255] NOT NULL,  
recording_id INT,  
frame_id INT,  
timestamp TIMESTAMP NOT NULL,  
PRIMARY KEY(recording_id, frame_id),  
FOREIGN KEY(recording_id) REFERENCES  
    Created_Documented_Recording(recording_id)  
    ON DELETE CASCADE,  
FOREIGN KEY(data) REFERENCES Contained_Analyzed_Frame2(data)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a Frame cannot exist without the original recording.

We used ON DELETE CASCADE for the data foreign key because a Frame cannot exist without the data in the Frame itself.

```
CREATE TABLE Contained_Analyzed_Frame2(  
data VARCHAR[255] PRIMARY KEY,  
resolution VARCHAR[255] NOT NULL)
```

```
CREATE TABLE Live(  
recording_id INT PRIMARY KEY,  
livestream_source VARCHAR[255] NOT NULL,  
    FOREIGN KEY (recording_id) REFERENCES  
        Created_Documented_Recording(recording_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a Live cannot exist without the original recording.

```
CREATE TABLE Video(  
recording_id INT PRIMARY KEY,  
duration INT NOT NULL,  
FOREIGN KEY (recording_id) REFERENCES Created_Documented_Recording  
    (recording_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a Video cannot exist without the original recording.

```
CREATE TABLE Documented_Saved_Transcript_1 (  
    transcript_id INT PRIMARY KEY,
```



```
recording_id INT,  
FOREIGN KEY (recording_id) REFERENCES Documented_Saved_Transcript_7  
(recording_id)  
ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a Transcript cannot exist without the original recording.

```
CREATE TABLE Documented_Saved_Transcript_2 (  
    transcript_id INT PRIMARY KEY,  
    user_id INT,  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (user_id) REFERENCES Calibrated_User (user_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_id foreign key because this relation cannot exist without the transcript.

We used ON DELETE CASCADE for the user_id foreign key because a Transcript cannot exist without the user who wants the transcript.

```
CREATE TABLE Documented_Saved_Transcript_3 (  
    transcript_id INT PRIMARY KEY,  
    transcript_data VARCHAR[255],  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_id foreign key because this relation cannot exist without the transcript.

```
CREATE TABLE Documented_Saved_Transcript_4 (  
    transcript_id INT PRIMARY KEY,  
    word_count INT  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE)
```

University of British Columbia, Vancouver

Department of Computer Science

We used ON DELETE CASCADE for the transcript_id foreign key because this relation cannot exist without the transcript.

```
CREATE TABLE Documented_Saved_Transcript_5 (  
    transcript_id INT PRIMARY KEY,  
    transcription_date date,  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_id foreign key because this relation cannot exist without the transcript.

```
CREATE TABLE Documented_Saved_Transcript_6 (  
    transcript_id INT PRIMARY KEY,  
    language VARCHAR[50],  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_id foreign key because this relation cannot exist without the transcript.

```
CREATE TABLE Documented_Saved_Transcript_7 (  
    recording_id INT PRIMARY KEY,  
    transcript_id INT,  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_id foreign key because this relation cannot exist without the transcript.

```
CREATE TABLE Documented_Saved_Transcript_8 (  
    recording_id INT PRIMARY KEY,  
    user_id INT,  
    FOREIGN KEY (recording_id) REFERENCES Documented_Saved_Transcript_7  
    (recording_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (user_id) REFERENCES Calibrated_User (user_id)  
    ON DELETE CASCADE)
```

University of British Columbia, Vancouver

Department of Computer Science

We used ON DELETE CASCADE for the recording_id foreign key because a transcript cannot exist without the original recording.

We used ON DELETE CASCADE for the user_id foreign key because a Transcript cannot exist without the user who wants the transcript.

```
CREATE TABLE Documented_Saved_Transcript_9 (  
    recording_id INT PRIMARY KEY,  
    transcript_data VARCHAR[255],  
    FOREIGN KEY (recording_id) REFERENCES Documented_Saved_Transcript_7  
    (recording_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (transcript_data) REFERENCES Documented_Saved_Transcript_10  
    (transcript_data)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a transcript cannot exist without the original recording.

We used ON DELETE CASCADE for the transcript_data foreign key because a transcript cannot exist without the content of the transcript.

```
CREATE TABLE Documented_Saved_Transcript_10 (  
    recording_id INT PRIMARY KEY,  
    word_count INT,  
    FOREIGN KEY (recording_id) REFERENCES Documented_Saved_Transcript_7  
    (recording_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a transcript cannot exist without the original recording.

```
CREATE TABLE Documented_Saved_Transcript_11 (  
    recording_id INT PRIMARY KEY,  
    transcription_date date,  
    FOREIGN KEY (recording_id) REFERENCES Documented_Saved_Transcript_7  
    (recording_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (transcript_data) REFERENCES Documented_Saved_Transcript_10  
    (transcript_data)  
    ON DELETE CASCADE)
```

University of British Columbia, Vancouver

Department of Computer Science

We used ON DELETE CASCADE for the recording_id foreign key because a transcript cannot exist without the original recording.

We used ON DELETE CASCADE for the transcript_data foreign key because a transcript cannot exist without the content of the transcript.

```
CREATE TABLE Documented_Saved_Transcript_12 (  
    recording_id INT PRIMARY KEY,  
    language VARCHAR[50],  
    FOREIGN KEY (recording_id) REFERENCES Documented_Saved_Transcript_7  
    (recording_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the recording_id foreign key because a transcript cannot exist without the original recording.

```
CREATE TABLE Documented_Saved_Transcript_13 (  
    transcript_data VARCHAR[255] PRIMARY KEY,  
    word_count INT,  
    FOREIGN KEY (transcript_data) REFERENCES Documented_Saved_Transcript_10  
    (transcript_data)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_data foreign key because a transcript cannot exist without the content of the transcript.

```
CREATE TABLE Documented_Saved_Transcript_14 (  
    transcript_data VARCHAR[255] PRIMARY KEY,  
    language VARCHAR[50],  
    FOREIGN KEY (transcript_data) REFERENCES Documented_Saved_Transcript1  
    (transcript_id)  
    ON DELETE CASCADE)
```

We used ON DELETE CASCADE for the transcript_data foreign key because a transcript cannot exist without the content of the transcript.

```
CREATE TABLE Translated_Word_1 (  
    instance_id INT PRIMARY KEY,  
    transcript_id INT,  
    FOREIGN KEY (transcript_id) REFERENCES Documented_Saved_Transcript_1  
    (transcript_id)  
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for transcript_id as all words must belong in the specific transcript tied to the specific recording uploaded by the user.

```
CREATE TABLE Translated_Word_2 (  
    instance_id INT PRIMARY KEY,  
    handmark_id INT,  
    FOREIGN KEY (handmark_id) REFERENCES Predicted_Gesture_Handmark1  
    (handmark_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (instance_id) REFERENCES Translated_Word_1 (instance_id)  
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for handmark_id as we cannot determine the translated word without being given a specific handmark from the specific recording, and it was used for translation as this relation cannot exist without the original translation key in Translated_Word_7.

```
CREATE TABLE Translated_Word_3 (  
    instance_id INT PRIMARY KEY,  
    character_length INT,  
    FOREIGN KEY (instance_id) REFERENCES Translated_Word_1 (instance_id)  
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for translation and instance_id as this relation cannot exist without the original keys in Translated_Word_7 and Translated_Word_1.

```
CREATE TABLE Translated_Word_4 (  
    instance_id INT PRIMARY KEY,  
    translation_confidence INT,  
    FOREIGN KEY (instance_id) REFERENCES Translated_Word_1 (instance_id)  
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for instance_id as this relation cannot exist without the original key in Translated_Word_1.

```
CREATE TABLE Translated_Word_5 (  
    instance_id INT PRIMARY KEY,  
    translation VARCHAR[255] NOT NULL,  
    FOREIGN KEY (translation) REFERENCES Translated_Word_7 (translation)  
    ON DELETE CASCADE,
```

```
FOREIGN KEY (instance_id) REFERENCES Translated_Word_1 (instance_id)
ON DELETE CASCADE,
UNIQUE (translation))
```

ON DELETE CASCADE was used for translation and instance_id as this relation cannot exist without the original keys in Translated_Word_7 and Translated_Word_1.

```
CREATE TABLE Translated_Word_6 (
    instance_id INT PRIMARY KEY,
    model_id INT,
    FOREIGN KEY (model_id) REFERENCES Trained_Machine_Learning_Model (model_id)
    ON DELETE CASCADE,
    FOREIGN KEY (instance_id) REFERENCES Translated_Word_1 (instance_id)
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for model_id as we cannot determine the translated word without using the specific model based on this specific user's definitions, and it was used for instance_id as this relation cannot exist without the original key in Translated_Word_1.

```
CREATE TABLE Translated_Word_7 (
    Translation VARCHAR[255] PRIMARY KEY,
    character_length INT)
```

```
CREATE TABLE Translated_Word_8 (
    model_id INT,
    handmark_id INT,
    translation VARCHAR[255],
    translation_confidence INT NOT NULL,
    PRIMARY KEY (model_id, handmark_id, translation),
    FOREIGN KEY (model_id) REFERENCES Trained_Machine_Learning_Model (model_id)
    ON DELETE CASCADE,
    FOREIGN KEY (handmark_id) REFERENCES
Predicted_Gesture_Handmark1(handmark_id)
    ON DELETE CASCADE,
    FOREIGN KEY (translation) REFERENCES Translated_Word_7(translation)
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for handmark_id as we cannot determine the translated word without being given a specific handmark from the specific recording, and it was used for

translation as this relation cannot exist without the original translation key in Translated_Word_7.

```
CREATE TABLE Translated_Word_9 (  
    handmark_id INT PRIMARY KEY,  
    word_timestamp TIMESTAMP NOT NULL,  
    FOREIGN KEY (handmark_id) REFERENCES  
    Predicted_Gesture_Handmark1(handmark_id)  
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for handmark_id as we cannot determine the translated word without being given a specific handmark from the specific recording.

```
CREATE TABLE Predicted_Gesture_Handmark1(  
    handmark_id INT PRIMARY KEY,  
    def_id INT,  
    model_id INT,  
    FOREIGN KEY (model_id) REFERENCES Trained_Machine_Learning_Model (model_id)  
    ON DELETE CASCADE,  
    FOREIGN KEY (def_id) REFERENCES Predicted_Gesture_Handmark2(def_id)  
    ON DELETE CASCADE)
```

ON DELETE CASCADE was used for model_id and def_id as we cannot predict the gesture without the specific model and definitions unique to the user.

```
CREATE TABLE Predicted_Gesture_Handmark2(  
    def_id INT PRIMARY KEY,  
    number_of_frames INT NOT NULL,  
    x_position VARCHAR[255] NOT NULL,  
    y_position VARCHAR[255] NOT NULL,  
    UNIQUE(x_position, y_position))
```

```
CREATE TABLE Trained_Machine_Learning_Model(  
    model_id INT PRIMARY KEY,  
    handmark_id INT,  
    accuracy INT,  
    hyperparameter INT NOT NULL,  
    model_type VARCHAR[255],  
    FOREIGN KEY (handmark_id) REFERENCES Predicted_Gesture_Handmark  
    (handmark_id))
```

The model does not depend on a single handmark, as it instead uses a training set of multiple. Therefore, deleting one handmark should not delete the entire trained model. As such, we are not using ON DELETE CASCADE for this table.

For our project, since Oracle doesn't support ON UPDATE, we'll implement our program in a way which doesn't allow the user to edit their initial details or make changes in a way which would require system-wide updates. Data will be inserted, but this will not result in changes to pre-existing data.

9. We did not use AI for any part of this assignment.